ViewModel и однонаправленный поток данных



Состояние экрана, события и списки

Разработка мобильных приложений на Android

От состояния в remember к состоянию экрана

Что должно стать понятным

- Почему состояние в remember теряется при повороте экрана
- Чем отличаются rememberSaveable, ViewModel и SavedStateHandle
- Как устроен однонаправленный поток данных
- Почему экран получает состояние и функции, а не ViewModel
- Как показать длинный список через LazyColumn

Результат: персонаж с инвентарём переживает поворот, а HeroScreen не знает о ViewModel.

Где мы остановились

```
class HeroStateHolder {
    var hp by mutableStateOf(100)
        private set

    fun onDamage() {
        hp = (hp - 10).coerceAtLeast(0)
    }
}

@Composable
fun HeroScreen() {
    val holder = remember { HeroStateHolder() }
    HealthPanel(holder.hp, holder::onDamage)
}
```

Обычный класс хранит состояние и правила.

private set: менять hp может только holder.

remember хранит сам объект holder.

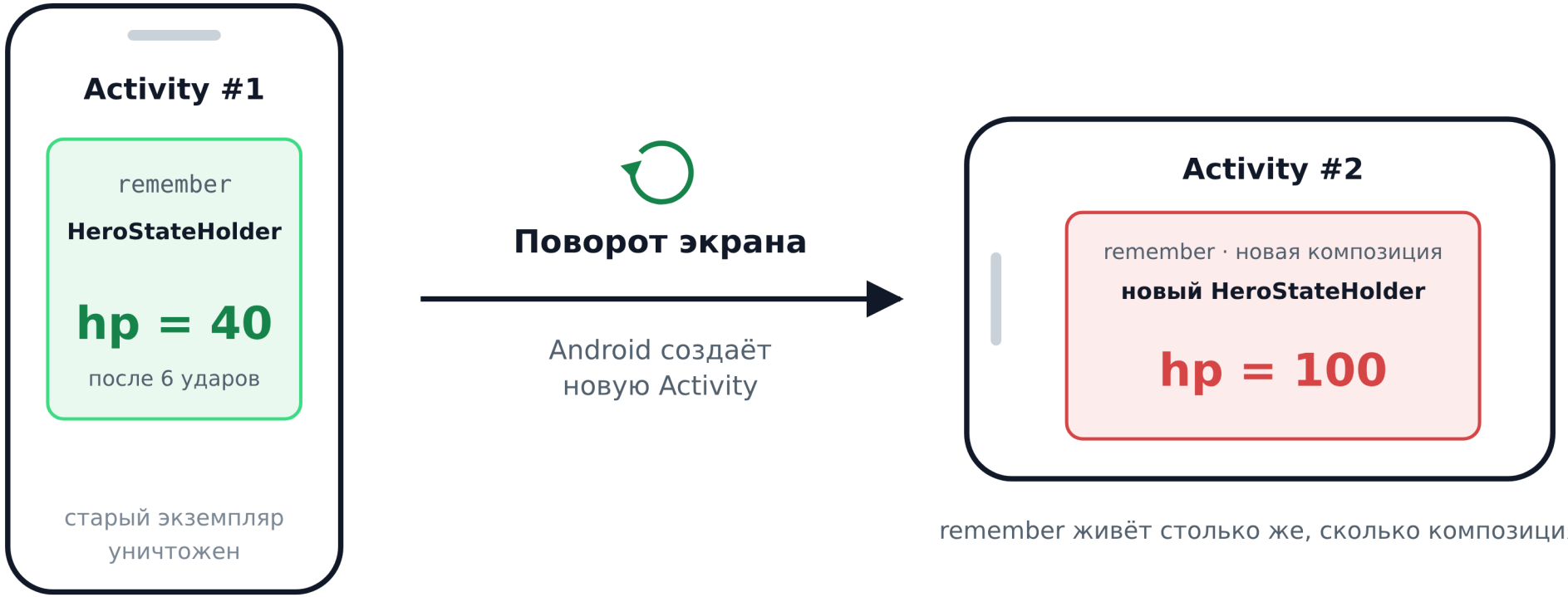
Состояние и логика уже вынесены из UI. Вопрос — сколько живёт holder.

Повернули телефон. Где здоровье?

Нанесли шесть ударов: hp = 40.
Повернули экран: снова 100.

Ничего не нажимали. Кто «сбросил» здоровье?

Поворот пересоздаёт Activity



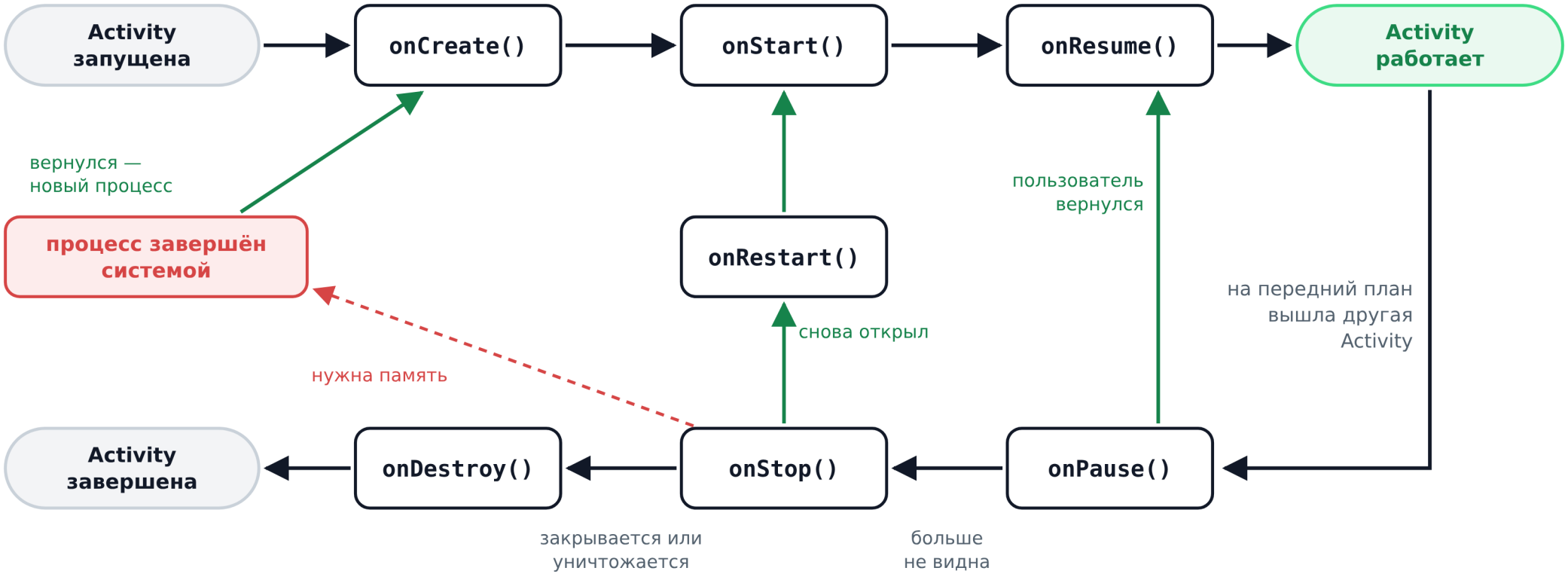
Новая Activity — новая композиция — новый remember — новый HeroStateHolder.

Изменение конфигурации

- Поворот экрана
- Тёмная тема, язык системы, размер шрифта
- Изменение размера окна: многооконный режим, складные устройства
- По умолчанию Activity уничтожается и создаётся заново

Пересоздание Activity — обычная ситуация, к которой приложение должно быть готово.

Жизненный цикл Activity



Поворот: onPause → onStop → onDestroy, затем новый экземпляр: onCreate → onStart → onResume.

rememberSaveable: переживает пересоздание

```
@Composable
fun HeroScreen() {
    var hp by rememberSaveable { mutableStateOf(100) }

    HealthPanel(
        hp = hp,
        onDamage = {
            hp = (hp - 10).coerceAtLeast(0)
        }
    )
}
```

Значение сохраняется в Bundle перед уничтожением Activity.

Новая композиция берёт его оттуда.

Int, String, Boolean — подходят.

rememberSaveable восстанавливает значение после поворота и завершения процесса.

rememberSaveable не хранит экран целиком

- В Bundle помещаются простые значения и Parcelable
- Для своего класса нужен Saver или @Parcelize
- Bundle рассчитан на небольшие данные
- Правила изменения снова оказываются внутри composable

Нужен владелец состояния экрана, который живёт дольше Activity.

ViewModel: владелец состояния экрана

- Класс из Jetpack Lifecycle для состояния и логики экрана
- Переживает изменения конфигурации
- Хранится у владельца: Activity, а позже — экрана навигации
- Уничтожается, когда владелец уходит окончательно

ViewModel — тот же state holder, но его временем жизни управляет Android.

Из HeroStateHolder в ViewModel

```
class HeroViewModel : ViewModel() {
    var hp by mutableStateOf(100)
        private set

    fun onDamage() {
        hp = (hp - 10).coerceAtLeast(0)
    }
}
```

Тот же класс.

Добавился родитель ViewModel().

Состояние и правила не изменились.

Меняется не код состояния, а то, кто создаёт объект и сколько он живёт.

viewModel() выдаёт объект

```
@Composable
fun HeroScreen() {
    val viewModel: HeroViewModel = viewModel()

    HealthPanel(
        hp = viewModel.hp,
        onDamage = viewModel::onDamage
    )
}
```

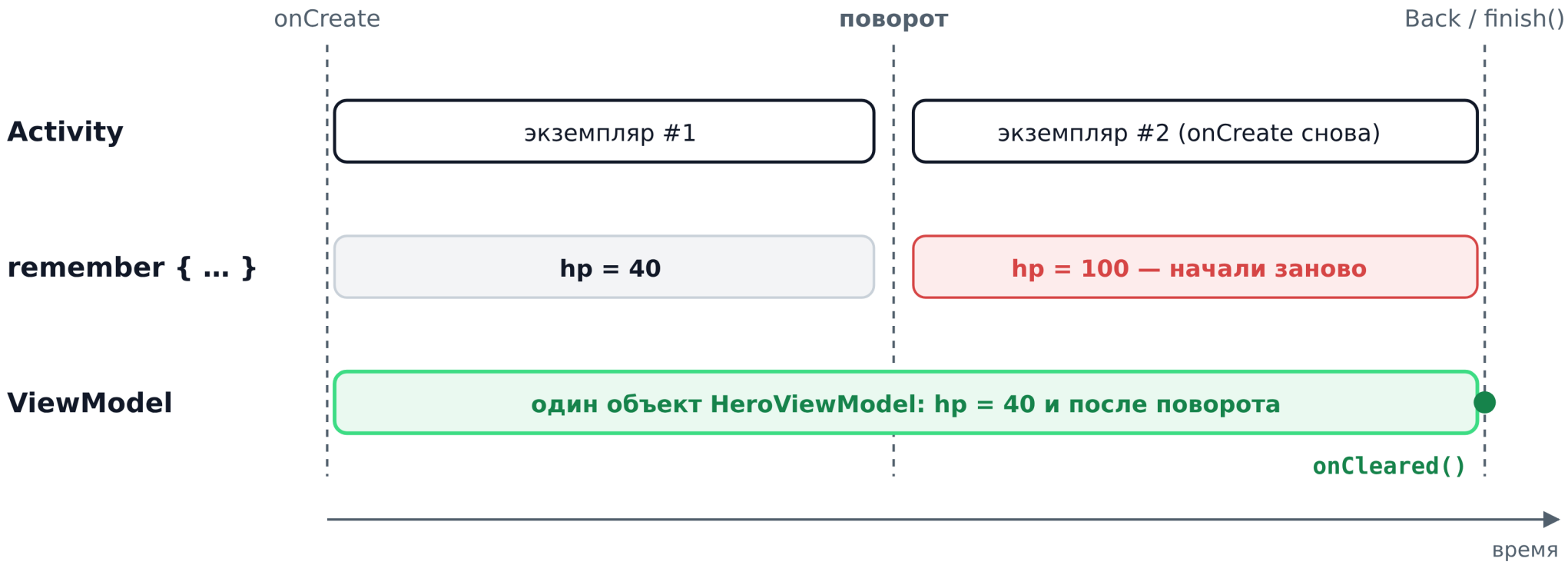
viewModel() ищет объект у Activity.

Первый вызов создаёт HeroViewModel.

После поворота — тот же объект.

Конструктор HeroViewModel() в composable мы не вызываем.

ViewModel живёт дольше Activity



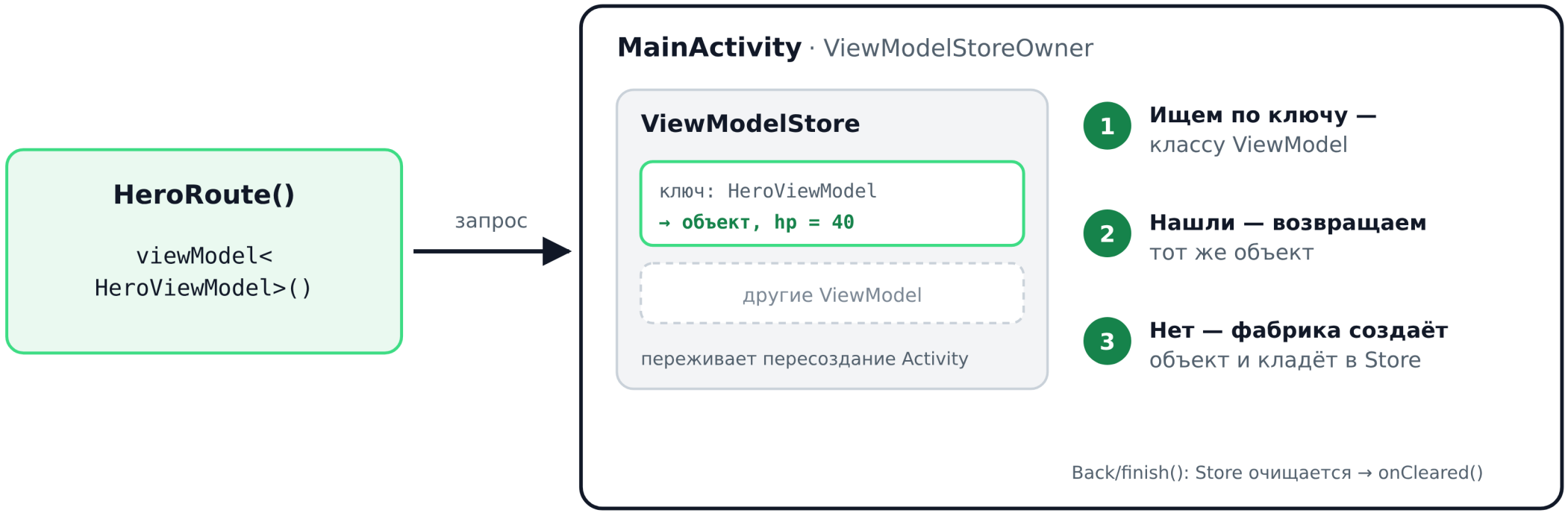
Поворот пересоздаёт Activity и remember, но не ViewModel.

Почему не remember { HeroViewModel() }

	remember { HeroViewModel() }	viewModel()
Где хранится объект	В композиции	В ViewModelStore владельца
После поворота	Новый объект, hp = 100	Тот же объект
onCleared()	Не вызывается	Вызывается при уходе владельца
Кто создаёт	Мы, конструктором	Фабрика по запросу

ViewModel не создают конструктором в composable — её выдаёт владелец.

Откуда viewModel() берёт объект



Один владелец — один объект на ключ. Ключ по умолчанию — класс ViewModel.

Когда ViewModel уничтожается

Событие	Что с HeroViewModel
Рекомпозиция	Ничего, тот же объект
Поворот, тема, язык	Тот же объект
Back или finish(): экран закрыт	onCleared(), объект удаляется
Система завершила процесс в фоне	Объект потерян, onCleared() не вызван

onCleared — для освобождения ресурсов, а не для сохранения данных.

Приложение долго было в фоне

**Системе понадобилась память,
процесс приложения завершён.**

Пользователь вернулся через список недавних приложений. Что он увидит: hp = 40 или 100?

SavedStateHandle: Bundle внутри ViewModel

```
class HeroViewModel(
    private val saved: SavedStateHandle
) : ViewModel() {

    var hp by mutableStateOf(
        saved.get<Int>("hp") ?: 100
    )
        private set

    fun onDamage() {
        hp = (hp - 10).coerceAtLeast(0)
        saved["hp"] = hp
    }
}
```

Фабрика по умолчанию сама передаёт SavedStateHandle.

При создании читаем сохранённое значение.

При изменении — записываем.

В SavedStateHandle — только небольшие простые значения, как в Bundle.

Как проверить восстановление

- Свернуть приложение кнопкой Home
- Завершить процесс: adb shell am kill <имя пакета>
- Вернуться через список недавних приложений
- Или включить Don't keep activities в параметрах разработчика

Смахнуть приложение из недавних — другой сценарий: там состояние не восстанавливается.

Кто что переживает

Событие	remember	rememberSaveable	ViewModel	SavedStateHandle
Рекомпозиция	**да**	**да**	**да**	**да**
Пересоздание Activity	нет	**да**	**да**	**да**
Процесс завершён системой	нет	**да**	нет	**да**
Пользователь закрыл экран	нет	нет	нет	нет

ViewModel — для состояния экрана. Сохранять — только небольшие значения.

ViewModel есть. Отдадим её экрану?

fun HeroScreen(viewModel: HeroViewModel)

Экран сам возьмёт всё, что нужно. Что может пойти не так?

Соблазн: передать ViewModel вниз

```
@Composable
fun HeroScreen(viewModel: HeroViewModel) {
    Column {
        Text("Здоровье: ${viewModel.hp}")
        HealthPanel(viewModel)
        Button(onClick = { viewModel.onReset() }) {
            Text("Заново")
        }
    }
}
```

HealthPanel получает весь ViewModel.

Каждый компонент видит все методы.

Для Preview нужен настоящий объект.

Компоненты снова знают, где и как хранится состояние.

Что мешает ViewModel в параметрах

Задача	ViewModel в параметре	Состояние и функции
Preview с hp = 0	Создать ViewModel и довести до нуля	Передать HeroUiState(hp = 0)
Повторное использование	Работает только с HeroViewModel	Данные может дать любой владелец
Понять, что умеет компонент	Читать весь класс ViewModel	Всё видно в параметрах
Проверить UI	Нужна логика и её зависимости	Достаточно подставить данные

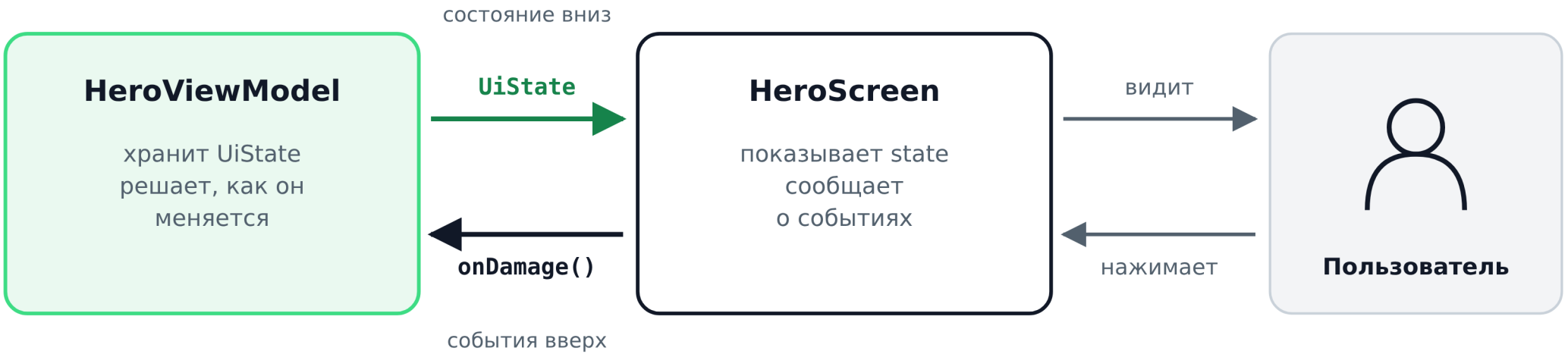
Документация Android: не передавайте ViewModel в другие composable-функции.

Однонаправленный поток данных

- Состояние хранится у одного владельца
- Состояние передаётся вниз, в UI
- UI сообщает о событиях вверх
- Только владелец решает, как изменить состояние

Знакомо по подъёму состояния. Теперь владелец — ViewModel.

Цикл UDF



1. нажал «Получить урон»
2. вызван onDamage()
3. новый UiState: hp = 90
4. экран показал 90

Нажатие не меняет экран напрямую: событие → новое состояние → новый UI.

UiState: снимок экрана

```
data class HeroUiState(
    val name: String = "Рыцарь",
    val hp: Int = 100,
    val maxHp: Int = 100,
) {
    val isDefeated: Boolean
        get() = hp == 0
}
```

Все данные экрана в одном объекте.

Только val.

Значения по умолчанию — начальный экран.

Экран показывает текущий UiState целиком.

ViewModel выдаёт новый UiState

```
class HeroViewModel : ViewModel() {
    var uiState by mutableStateOf(HeroUiState())
        private set

    fun onDamage() {
        val hp = (uiState.hp - 10).coerceAtLeast(0)
        uiState = uiState.copy(hp = hp)
    }

    fun onHeal() {
        val hp = minOf(uiState.hp + 10, uiState.maxHp)
        uiState = uiState.copy(hp = hp)
    }
}
```

private set: заменить uiState может только ViewModel.

copy создаёт новый снимок с другим hp.

Остальные поля копируются как есть.

onReset() — так же, через copy.

Снаружи uiState только читают. Изменения — через функции ViewModel.

Почему copy, а не var hp

```
// А: меняем поле внутри объекта
data class HeroUiState(var hp: Int = 100)

uiState.hp -= 10        // Compose не узнает


// Б: заменяем объект целиком
data class HeroUiState(val hp: Int = 100)

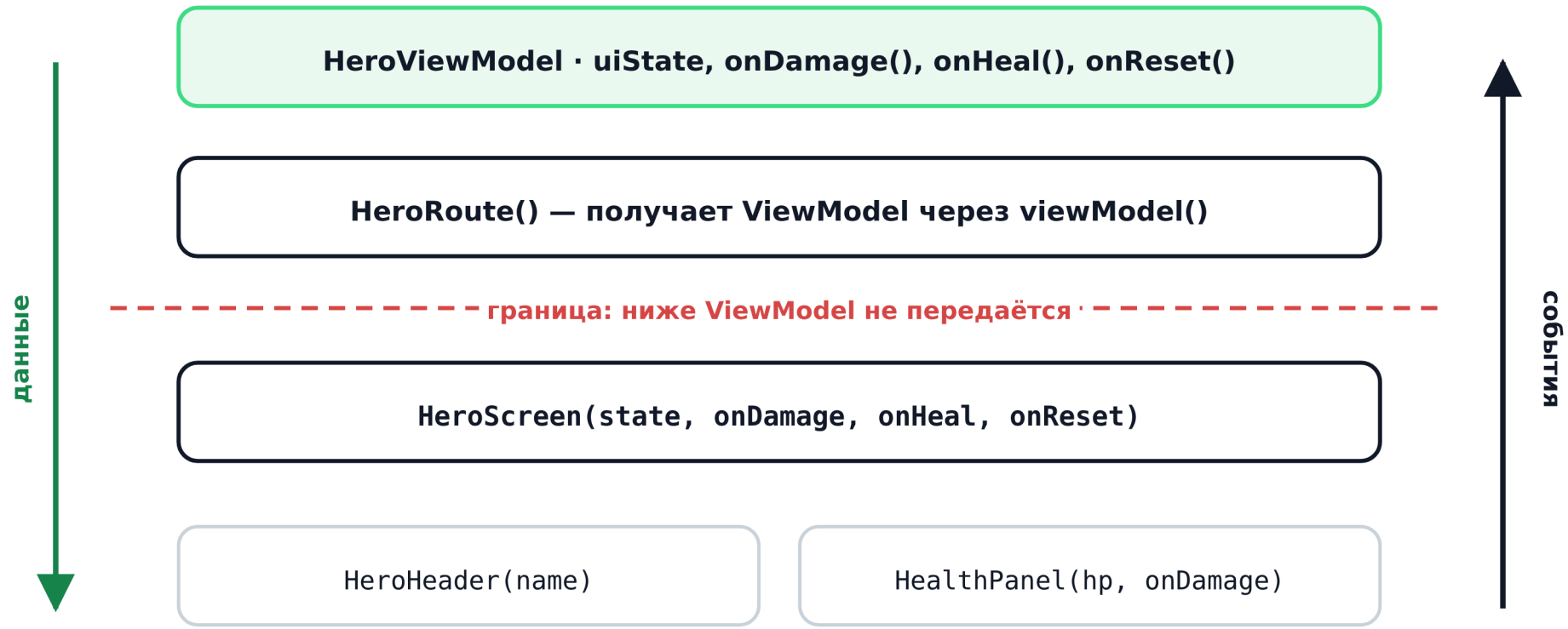
uiState = uiState.copy(hp = uiState.hp - 10)
```

mutableStateOf следит за заменой значения, а не за полями внутри него.

В варианте А менять hp может кто угодно.

Неизменяемый UiState: каждое изменение видно и происходит в одном месте.

Route и Screen: где проходит граница



ViewModel знает только Route. Всё, что ниже, получает данные и функции.

HeroRoute: единственное место с ViewModel

```
@Composable
fun HeroRoute(
    viewModel: HeroViewModel = viewModel()
) {
    HeroScreen(
        state = viewModel.uiState,
        onDamage = viewModel::onDamage,
        onHeal = viewModel::onHeal,
        onReset = viewModel::onReset,
    )
}
```

Route получает ViewModel.

Передаёт дальше снимок и функции.

::onDamage — ссылка на функцию.

В MainActivity: setContent { HeroTheme { HeroRoute() } }.

HeroScreen не знает о ViewModel

```
@Composable
fun HeroScreen(
    state: HeroUiState,
    onDamage: () -> Unit,
    onHeal: () -> Unit,
    onReset: () -> Unit,
) {
    Column {
        HeroHeader(name = state.name)
        HealthPanel(hp = state.hp, onDamage = onDamage)
        Button(onClick = onHeal) { Text("Лечить") }
        Button(onClick = onReset) { Text("Заново") }
    }
}
```

Получает снимок и обработчики.

Ни одного упоминания ViewModel.

HealthPanel — из прошлой лекции.

HeroScreen — stateless: всё нужное приходит параметрами.

Preview без ViewModel

```
@Preview(showBackground = true)
@Composable
fun DefeatedHeroPreview() {
    MaterialTheme {
        HeroScreen(
            state = HeroUiState(hp = 0),
            onDamage = {},
            onHeal = {},
            onReset = {},
        )
    }
}
```

Любое состояние экрана — одной строкой.

Логика и ViewModel не нужны.

Если экран легко показать в Preview, граница проведена правильно.

У персонажа появился инвентарь

В инвентаре 30 зелий.
Положим их в Column?

На экран помещается шесть строк. Нужно ли создавать остальные 24 сразу?

LazyColumn: ленивый список

- Компонует только строки, которые видны на экране
- Строки, ушедшие за экран, покидают композицию
- Содержимое описывается через item { } и items(...)
- Прокрутка встроена, verticalScroll не нужен

Column — для нескольких элементов. LazyColumn — для списков любой длины.

Что видно — то и в композиции



Рамка — экран.

Column создаёт все строки сразу, даже невидимые.

LazyColumn — только видимые. Ушедшие за экран покидают композицию.

Цена LazyColumn зависит от размера экрана, а не от длины списка.

Инвентарь — часть UiState

```
data class Item(
    val id: Int,
    val name: String,
    val heal: Int,
)

data class HeroUiState(
    val hp: Int = 100,
    val maxHp: Int = 100,
    val inventory: List<Item> = emptyList(),
)

fun startItems() = List(30) { i ->
    Item(i, "Зелье ${i + 1}", heal = 10 * (i % 3 + 1))
}
```

List, а не MutableList.

id — устойчивый идентификатор предмета.

Данные пока просто в памяти.

name — как раньше.

Список — такое же поле снимка, как hp.

Событие с данными: какой предмет

```
class HeroViewModel : ViewModel() {
    var uiState by mutableStateOf(
        HeroUiState(inventory = startItems())
    )
        private set

    fun onUseItem(id: Int) {
        val item = uiState.inventory
            .find { it.id == id } ?: return
        val hp = uiState.hp + item.heal
        uiState = uiState.copy(
            hp = hp.coerceAtMost(uiState.maxHp),
            inventory = uiState.inventory - item,
        )
    }
}
```

Экран сообщает id, ViewModel находит предмет.

inventory - item — новый список без предмета.

Старый список не меняется.

Правило «вылечить и убрать предмет» живёт только во ViewModel.

Inventory: LazyColumn

```
@Composable
fun Inventory(
    inventory: List<Item>,
    onUse: (Int) -> Unit,
    modifier: Modifier = Modifier,
) {
    LazyColumn(modifier) {
        items(inventory, key = { it.id }) { item ->
            ItemRow(item, onClick = { onUse(item.id) })
        }
    }
}
```

items — не цикл, а описание строки для каждого предмета.

key связывает строку с предметом.

onUse получает id.

Inventory тоже не знает о ViewModel: список и функция приходят параметрами.

ItemRow: одна строка

```
@Composable
fun ItemRow(item: Item, onClick: () -> Unit) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(16.dp)
    ) {
        Text(item.name, modifier = Modifier.weight(1f))
        Text("+${item.heal} hp")
    }
}
```

clickable делает всю строку кнопкой.

weight(1f): имя занимает всё свободное место.

Обычный Row из лекции 04.

Строка списка — обычный stateless-компонент.

Зачем items нужен key

	Без key	key = { it.id }
Строка определяется	Позицией в списке	id предмета
Удалили предмет выше	remember строки переезжает к соседу	Состояние остаётся у своего предмета
Список изменился	Прокрутка может сместиться	Позиция держится за видимый предмет
Анимация animateItem()	Не работает	Работает

key — уникальный и устойчивый: id предмета, а не индекс.

Список на экране персонажа

```
@Composable
fun HeroScreen(
    state: HeroUiState,
    onDamage: () -> Unit,
    onUseItem: (Int) -> Unit,
) {
    Column(Modifier.fillMaxSize()) {
        HealthPanel(hp = state.hp, onDamage = onDamage)
        Inventory(
            inventory = state.inventory,
            onUse = onUseItem,
            modifier = Modifier.weight(1f),
        )
    }
}
```

weight(1f): список занимает оставшуюся высоту.

В HeroRoute добавится ещё одна ссылка на функцию ViewModel.

LazyColumn не кладём в Column с verticalScroll: высота списка станет бесконечной.

Когда событий становится много

```
sealed interface HeroAction {
    data object Damage : HeroAction
    data object Heal : HeroAction
    data class UseItem(val id: Int) : HeroAction
}

@Composable
fun HeroScreen(
    state: HeroUiState,
    onAction: (HeroAction) -> Unit,
)
```

Один параметр вместо многих.

ViewModel разбирает действие через when.

По умолчанию — отдельные лямбды: они явно показывают, что умеет экран.

Кто за что отвечает

Часть	Отвечает за
HeroViewModel	Хранит UiState и правила его изменения
HeroUiState	Снимок данных, нужных экрану
HeroRoute	Получает ViewModel и связывает её с экраном
HeroScreen	Показывает state и сообщает о событиях
HealthPanel и другие	Части экрана, получают только параметры

У каждой части одна причина для изменения.

Не всё состояние нужно ViewModel

Во ViewModel

Данные экрана:
имя, здоровье.

Правила: урон,
лечение, границы.

То, что нужно логике.

Рядом с UI

Раскрыт ли блок
с подсказкой.

Прокрутка списка,
анимации, фокус.

То, что нужно одному компоненту.

Значение нужно логике экрана — во ViewModel. Только отображению — рядом с UI.

Частые ошибки

Симптом	Что проверить
После поворота hp снова 100	Не создан ли ViewModel через remember или конструктор
Нажатие не меняет экран	Заменяется ли uiState через copy
Preview не строится	Не передаётся ли ViewModel в HeroScreen
UI меняет hp в обход правил	Есть ли private set и val в UiState
Утечка памяти после поворота	Не хранит ли ViewModel Activity, View или Context
Удалили предмет, список прежний	Не меняется ли MutableList вместо нового списка
Состояние строки «перескочило»	Задан ли key у items

Сначала спрашиваем: кто владеет состоянием и сколько он живёт?

Проверяем понимание

- Почему HeroStateHolder в remember теряет hp при повороте?
- Что из сохранённого переживёт завершение процесса?
- Почему viewModel(), а не HeroViewModel() в composable?
- Где в нашем коде граница, ниже которой ViewModel нет?
- Почему uiState.copy(...), а не uiState.hp -= 10?
- Чем LazyColumn отличается от Column и зачем items нужен key?

Ответ должен объяснять поведение программы, а не только название API.

Теперь смотрим целое приложение

- Тот же персонаж, теперь с HeroViewModel
- Поворот экрана: здоровье на месте
- Завершение процесса: что восстановилось
- Инвентарь: прокрутка и использование зелий
- HeroScreen в Preview в разных состояниях



Сначала показываем готовый проект, затем переносим состояние вместе.

Практика: персонаж во ViewModel

- Перенести имя, здоровье и инвентарь в HeroUiState и HeroViewModel
- Разделить экран на HeroRoute и HeroScreen, ViewModel ниже не идёт
- Инвентарь на LazyColumn с key, нажатие использует предмет
- Preview: полное здоровье, «Персонаж побеждён», пустой инвентарь
- Здоровье переживает поворот и завершение процесса

Проверить: поворот, adb shell am kill и Preview без ViewModel.

Что дальше

- Корутины и viewModelScope: работа без блокировки UI
- StateFlow и collectAsStateWithLifecycle
- Загрузка списка: состояния загрузки и ошибки
- Навигация: список → карточка предмета

Схема ViewModel → UiState → Screen останется. Изменится источник данных.

Материалы для повторения

- Android Developers: The activity lifecycle, ViewModel overview
- Saved State module for ViewModel, Save UI states
- Where to hoist state, State holders and UI state
- UI layer: unidirectional data flow; Lists and grids

Ссылки на соответствующие темы есть в заметках к слайдам.